Result:

```
(base) yuvalpaz@mac ~ %  /usr/bin/env /Library/Java/JavaVirtualMachines/temurin-21.jdk/Contents/Home/bin/ja
va -XX:+ShowCodeDetailsInExceptionMessages -cp /private/var/folders/05/q4ct51216g30hgmr8mpd6kc80000gn/T/vsc
odesws_0de59/jdt_ws/jdt.ls-java-project/bin Week9Exercises
Input:[5, 3, 25, 7, 63, 4, 73, 42, 72, 3, 64, 6, 0, -34, 2, -521, 32]
Output:[3, 63, 42, 72, 3, 6, 0]

Input:[32.1, 2.76, 3.0, 1.45, 89.2]
Output: [32, 2, 3, 1, 89]

hello
apple
world
carrot
java
cow
fig
programming

Input:[5, 3, 25, 7, 63, 4, 73, 42, 72, 3, 64, 6, 0, -34, 2, -521, 32]
Output (>50 and divisible by 3): [63, 72]

Input:[5, -3, 15, 7, 3]
Output(Sum of squares): 317

Input:[Mary, Jane]
Output: [Hello, Mary, Hello, Jane]

Input:[5, -3, 15, 7, 3]
Output (cubed and prefixed with Result): [Result: 125, Result: -27, Result: 3375, Result: 343, Result: 27]

List generated by Supplier: [54, 80, 51, 9, 10, 95, 61, 78, 33, 100]

Input: {Hello, Dear}
Output: Hello Dear
Input: [Would, this, darkness, ever, lift?]
Output: Would this darkness ever lift?

InputList: [amy, Peter, Pam, Paul, Neela, Porter]
Filtered List (starts with 'P'): [Peter, Pam, Paul, Porter]
Filtered List (length > 4): [Peter, Neela, Porter]
```

Code:

```java
import java.util.Arrays;
import java.util.List;
import java.util.stream.*;
import java.util.function.*;
import java.util.*;


public class Week9Exercises {


   // Method for Exercise 10
   public static List<String> filterStrings(List<String> list, Predicate<String>
predicate) {
       return list.stream()
               .filter(predicate)
               .collect(Collectors.toList());
```

```
    }

    public static void main(String[] args) {
        List<Integer> ints = Arrays.asList(5, 3, 25, 7, 63, 4, 73, 42, 72, 3, 64, 6, 0,
-34, 2, -521, 32);
        List<String> strings = Arrays.asList("HELLO", "aPpLe", "WORLD", "carrot",
"JAVA", "cow", "fig", "PROGRAMMING");
        List<Double> doubles = List.of(32.1, 2.76, 3.0, 1.45, 89.2);
        List<String> namesList = List.of("Mary", "Jane");
        List<Integer> ints2 = Arrays.asList(5, -3, 15, 7, 3);

        /*
         * 1. Use a Predicate to filter a list of integers, keeping only those that are
         * divisible by 3.
         */
        Predicate<Integer> divBy3 = n -> n % 3 == 0;
        List<Integer> integers = ints.stream()
                .filter(divBy3)
                .collect(Collectors.toList());
        System.out.println("Input:" + ints);
        System.out.println("output:" + integers);
        System.out.println();

        /*
         * 2. Create a Function that takes a double and returns the whole number
portion.
         */
        Function<Double, Integer> intPart = d -> d.intValue();
        List<Integer> wholeParts = doubles.stream()
                .map(intPart)
                .collect(Collectors.toList());
        System.out.println("Input:" + doubles);
        System.out.println("Output: " + wholeParts);
        System.out.println();

        /* 3. Use a Consumer to print each string in a list in lowercase. */
        Consumer<String> printLower = s -> System.out.println(s.toLowerCase());
        strings.forEach(printLower);
        System.out.println();

        /*
```

```
        * 4. Write code that filters a list of integers to keep only those greater
than
        * 50 and divisible by 3. Combine multiple predicates and reuse your predicate
        * from 1.
        */
       System.out.println("Input:" + ints);
       Predicate<Integer> greaterThan50 = n -> n > 50;
       Predicate<Integer> combined = greaterThan50.and(divBy3);
       List<Integer> filtered = ints.stream()
               .filter(combined)
               .collect(Collectors.toList());
       System.out.println("Output (>50 and divisible by 3): " + filtered);
       System.out.println();

       /*
        * 5. Write code to square each number in a list of integers and then calculate
        * the sum of all squared values.
        */
       System.out.println("Input:" + ints2);
       int sumOfSquares = ints2.stream()
               .mapToInt(n -> n * n)
               .sum();
       System.out.println("Output(Sum of squares): " + sumOfSquares);
       System.out.println();

       /*
        * 6. Write a function to add a prefix "Hello, " to a given string.
        * Apply it to a list of names.
        */
       System.out.println("Input:" + namesList);
       Function<String, String> addHello = s -> "Hello, " + s;
       List<String> greetedNames = namesList.stream()
               .map(addHello)
               .collect(Collectors.toList());
       System.out.println("Output: " + greetedNames);
       System.out.println();

       /*
        * 7. Write a function that converts a number to its cube.
        * Write a function that converts a number to a string prefixed by "Result: ".
        * Chain these functions and apply them to a list of numbers.
        */
```

```java
        Function<Integer, Integer> cube = n -> n * n * n;
        Function<Integer, String> addResultPrefix = n -> "Result: " + n;
        Function<Integer, String> cubeAndPrefix = cube.andThen(addResultPrefix);

        List<String> result = ints2.stream()
                .map(cubeAndPrefix)
                .collect(Collectors.toList());
        System.out.println("Input:" + ints2);
        System.out.println("Output (cubed and prefixed with Result): " + result);
        System.out.println();

        /*
         * 8. Write a Supplier to generate random integers between 1 and 100.
         * Call the supplier 10 times and create a list of the generated values.
         */
        Random random = new Random();
        Supplier<Integer> randomInt = () -> random.nextInt(100) + 1;
        List<Integer> intList = Stream.generate(randomInt)
                .limit(10)
                .collect(Collectors.toList());
        System.out.println("List generated by Supplier: " + intList);
        System.out.println();

        /*
         * 9. Create a BiFunction that takes two strings and returns their
concatenation
         * with a space in between. Use it on pairs of strings.
         */
        BiFunction<String, String, String> concatWithSpace = (a, b) -> a + " " + b;
        System.out.println("Input: {Hello, Dear}");
        System.out.println("Output: " + concatWithSpace.apply("Hello", "Dear"));

        List<String> strList2 = List.of("Would", "this", "darkness", "ever", "lift?");
        String newStr = strList2.stream()
                .reduce("", (a, b) -> a.isEmpty() ? b : concatWithSpace.apply(a, b));
        System.out.println("Input: " + strList2);
        System.out.println("Output: " + newStr);
        System.out.println();

        /*
         * 10. Write a method that takes a list of strings and a Predicate<String>.
         * The method returns a new list containing only strings that satisfy the
```

```
         * predicate. Test with:
         * (a) Strings that start with "P"
         * (b) Strings longer than 4 characters
         */
        List<String> list3 = List.of("amy", "Peter", "Pam", "Paul", "Neela", "Porter");
        System.out.println("InputList: " + list3);

        // (a) Strings starting with "P"
        List<String> list3Filtered = filterStrings(list3, s -> s.startsWith("P"));
        System.out.println("Filtered List (starts with 'P'): " + list3Filtered);

        // (b) Strings longer than 4 characters
        List<String> list3FilteredLong = filterStrings(list3, s -> s.length() > 4);
        System.out.println("Filtered List (length > 4): " + list3FilteredLong);
        System.out.println();
    }
}
```